

AI Data-Residency Fixes — Prompts

iSCARB Platform | Prepared for the Backend Developer

Two code-agent prompts: what they fix, how, and why

This document contains **two prompts** to give the coding agent. They came out of a code-grounded data-flow audit of how the platform uses NVIDIA for AI. Each prompt below is preceded by **what it fixes**, **how it fixes it**, and **why it matters**, so the reasoning is clear before the change is made.

Context (from the audit): Almost all AI traffic goes to NVIDIA's hosted API at `integrate.api.nvidia.com`. Student answer text and faculty source excerpts are sent in prompts. Structured identifiers (name/email/ID) are not placed in prompts by the builders — but there is **no PII-stripping step**, and the primary AI path **hardcodes** the NVIDIA URL so it cannot be redirected to an in-Kingdom endpoint via env alone.

Prompt 1 — Add a PII-Stripping Layer Before External AI Calls

What it fixes

- There is currently **no sanitization** of prompt content before it is sent to the external NVIDIA API. If a student types their name, email, national ID, or student ID **inside** a free-text answer, that identifier is transmitted unchanged out of the Kingdom.

How it fixes it

- Introduce a single shared sanitization function that runs on all user-derived prompt content immediately before it is sent to any external AI endpoint.
- It detects and redacts common personal identifiers (emails, phone numbers, national ID / Iqama-style numbers, student IDs, and obvious name patterns where feasible) from the answer/source text, replacing them with neutral placeholders (e.g. `[REDACTED_EMAIL]`).
- It is applied centrally at the AI-client boundary (in the shared `chatJson` / `chatJsonRaw` path) so every call site is covered and none can bypass it.
- Scoring correctness is preserved — redaction targets identifiers, not the substantive answer content the rubric evaluates.

Why it matters

- It materially reduces PDPL exposure on **both** the current hosted-NVIDIA path and any future setup — personal identifiers a student embeds in text no longer leave the Kingdom.
- It is a small, self-contained change with high compliance value, and is worth doing regardless of the hosting/region decision.

Prompt to paste:

Role: You are adding a centralized PII-stripping (sanitization) layer that runs on all user-derived content before it is sent to any external AI endpoint. The data-flow audit confirmed the app sends student answer text and faculty source excerpts to NVIDIA's hosted API with no sanitization, so identifiers a user types into free text leave the app unchanged. Fix this centrally. Read the current code as the source of truth; change only what is needed.

Implement:

1. Create a single shared sanitizer (e.g. `src/lib/ai/pii-sanitizer.ts`) that takes a string and returns it with common personal identifiers redacted: email addresses, phone numbers, national ID / Iqama-style number patterns, student IDs, and clearly-formatted identifiers. Replace each with a neutral placeholder (e.g. `[REDACTED_EMAIL]`, `[REDACTED_ID]`). Do not alter the substantive answer content.
2. Apply it at the AI-client boundary – in the shared `chatJson` / `chatJsonRaw` path in `src/lib/ai-engine.ts` (and the secondary clients) – so user-derived prompt content is sanitized immediately before the request is sent to the external endpoint. Every call site must be covered; none may bypass it.
3. Do NOT sanitize non-personal structural content (rubric, scenario, model instructions). Target only the user/student/faculty-derived free text.
4. Keep scoring/generation quality intact – redaction removes identifiers, not the meaning the rubric evaluates.

Verify with real evidence: craft a test answer containing a fake email, phone, national ID, and student ID; confirm the payload actually sent to the AI endpoint has them redacted (log/inspect the outgoing body). Confirm a normal answer with no identifiers is unchanged and still scores correctly. Confirm the sanitizer is applied on every AI call path, not just one. Paste the before/after of a sanitized payload.

Constraints: centralized (one sanitizer, applied at the boundary), no bypass, scoring preserved, no leaking of the correct-answer key to the client (unchanged). Report only what you changed.

Prompt 2 — Make the NVIDIA Endpoint Configurable (In-Kingdom Ready)

What it fixes

- The primary AI path **hardcodes** `https://integrate.api.nvidia.com/v1/chat/completions` in `src/lib/ai-engine.ts` and does **not** read `NVIDIA_BASE_URL` / `OPENAI_BASE_URL`. As a result, the app cannot be pointed at a self-hosted in-Kingdom NIM endpoint without a code change — setting the env var alone does nothing for the dominant traffic.

How it fixes it

- Parameterize the base URL: `getClient() / chatJson / chatJsonRaw` read `process.env.NVIDIA_BASE_URL` || `OPENAI_BASE_URL` and fall back to the current hosted default — matching the pattern the secondary clients (`llm-verifier.ts`, `nvidia-vlm-gate.ts`) already use.
- Do the same for the hardcoded image-generation URL in `generateImage()` if image gen is used.
- Keep the existing default behavior identical (still hosted NVIDIA) when no env override is set — so nothing changes for the current deployment; it only becomes **possible** to redirect later.

Why it matters

- It turns the future move to fully in-Kingdom AI (self-hosted NIM) into a **config change** instead of a code change under pressure. When compliance or scale requires it, the dev flips an env var to point at an in-Kingdom NIM.
- Zero behavior change now (safe): the default remains the hosted API. This is preparation, not a switch.
- Low effort, high option-value — it de-risks the compliance roadmap without committing to GPU costs today.

Prompt to paste:

Role: You are making the NVIDIA AI endpoint configurable so the app can later be pointed at a self-hosted in-Kingdom NIM without a code rewrite. Right now the primary AI path hardcodes the NVIDIA URL and ignores the base-URL env vars, so it cannot be redirected. This change must NOT alter current behavior — the hosted NVIDIA API stays the default. Read the current code as truth; change only what is needed.

Implement:

1. In `src/lib/ai-engine.ts`, change `getClient() / chatJson / chatJsonRaw` so the request base URL is read from `process.env.NVIDIA_BASE_URL` || `process.env.OPENAI_BASE_URL`, falling back to the current default `https://integrate.api.nvidia.com/v1` — mirroring the pattern already used in `src/lib/visual/llm-verifier.ts` and `src/lib/lecture/visual/nvidia-vlm-gate.ts`. Build the chat URL as `${base}/chat/completions`.
2. Do the same for the hardcoded image URL in `generateImage()` (parameterize the base), if image generation is used.
3. Leave auth (Bearer key round-robin) working as-is; ensure a custom base URL can use the provided key.
4. Do NOT change the default behavior: with no env override set, the app must behave exactly as today (hosted NVIDIA). This is enablement only, not a switch.

Verify with real evidence: with `NVIDIA_BASE_URL` unset, confirm calls still go to `integrate.api.nvidia.com` and the app behaves identically. Then set `NVIDIA_BASE_URL` to a test/staging URL and confirm the primary AI path (scoring, generation) actually targets that URL (log/inspect the outgoing request target). Confirm the secondary clients still work. Paste evidence of both (unset = hosted default; set = redirected).

Constraints: no behavior change when unset; centralized through the shared helpers so all callers benefit; do not break auth, model selection, timeouts, or concurrency limits. Report only what you changed.

Suggested order

- **Prompt 1 (PII-stripping)** first — it improves compliance immediately on the current hosted-NVIDIA setup.
- **Prompt 2 (configurable endpoint)** next — it prepares the in-Kingdom NIM migration path with zero behavior change now.

Both prompts are report-only in effect on current behavior: Prompt 1 adds protection, Prompt 2 adds capability without changing defaults. Neither requires the hosting/region decisions to be finalized first — they are safe to apply now.